Практичне заняття №6

- **Тема:** Мікроконтролерний вузол: GPIO, АЦП, таймер, UART
- **Тривалість:** 2 год
- **Інструмент:** *Wokwi* (браузерний симулятор Arduino/ESP32), платформа Arduino Uno

Мета

Закріпити практичні навички роботи з цифровими та аналоговими периферійними інтерфейсами мікроконтролера: організацію цифрового вводу-виводу через GPIO, зчитування показань аналогового датчика через АЦП, побудову неблокувальної періодичної логіки на основі millis() та формування структурованого повідомлення для виводу через UART. Формат повідомлення, розроблений у цій роботі, буде повторно використано в практичному занятті №10 для передавання даних мережею, тому варто підійти до його проєктування свідомо.

Теоретичні відомості

Порти GPIO мікроконтролера можуть працювати в режимах цифрового входу або виходу; режим налаштовується функцією pinMode(), зчитування стану виконується digitalRead(), а встановлення стану виходу - digitalWrite(). Для кнопок без зовнішнього резистора зручно використовувати режим INPUT_PULLUP. **Аналого-цифровий перетворювач (АЦП)** дає змогу отримати числове значення аналогової напруги; розрядність АЦП Arduino Uno - **10 біт (0-1023)**. Широтно-імпульсна модуляція (ШІМ, *PWM*) реалізується функцією analogWrite() і дає змогу емулювати аналоговий рівень сигналу шляхом зміни шпаруватості імпульсів. У вбудованих системах **критично важливо уникати блокувальних затримок**: delay() зупиняє виконання всієї програми на заданий час, тоді як побудова періодичних дій на основі millis() дозволяє програмі одночасно обробляти кілька задач без блокування. Виведення результатів роботи вузла на комп'ютер здійснюється через **UART**, який у середовищі Arduino реалізується об'єктом Serial (Serial.begin(), Serial.print(), Serial.println()). Детальніше - Лекція №11.

Хід роботи

1. **Створіть** проєкт у *Wokwi*, оберіть Arduino Uno. **Підключіть** кнопку з підтягувальним резистором і світлодіод; реалізуйте перемикання стану на кожне натискання. 2. **Підключіть** аналоговий датчик (потенціометр за варіантом), зчитайте analogRead() і перетворіть у змістовну величину через map().
3. **Організуйте** на основі millis() (без delay()) періодичне зчитування раз на секунду і вивід рядка в Serial у форматі свого варіанта.
4. **Запустіть** симуляцію, перевірте кнопку і формат виводу, зробіть **скріншот** схеми і монітора порту.

```
const int buttonPin = 2, ledPin = 8, sensorPin = A0;
bool ledState = false, lastButtonState = HIGH;
unsigned long lastMeasurement = 0, packetId = 0;
const unsigned long interval = 1000;

void setup() {
  pinMode(buttonPin, INPUT_PULLUP);
  pinMode(ledPin, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  bool buttonState = digitalRead(buttonPin);
  if (buttonState == LOW && lastButtonState == HIGH) {
    ledState = !ledState;
```

```
        digitalWrite(ledPin, ledState);
    }
    lastButtonState = buttonState;

    unsigned long now = millis();
    if (now - lastMeasurement >= interval) {
        lastMeasurement = now;
        int value = map(analogRead(sensorPin), 0, 1023, -20, 40);
        Serial.print("id="); Serial.print(packetId++);
        Serial.print(";t="); Serial.print(now);
        Serial.print(";value="); Serial.println(value);
    }
}
```

Індивідуальні варіанти

№	Сенсор (імітація)	Формат повідомлення	Приклад рядка виводу
1	Температура	CSV з роздільником ;	12345;temp;23
2	Освітленість	JSON-подібний рядок	{"sensor":"light","value":540}
3	Вологість	формат ключ=значення	sensor=hum;value=61
4	Рівень рідини	поля фіксованої ширини	LVL | 74|
5	Відстань	з контрольною сумою	dist=120;chk=45
6	Температура	з міткою часу millis()	t=8032;temp=19
7	Освітленість	з порядковим номером пакета	seq=7;light=310
8	Вологість	з прапорцем перевищення порогу	hum=88;alarm=1
9	Рівень рідини	з ковзним середнім останніх N вимірів	level=45;avg=43
10	Відстань	з мінімумом/максимумом за період	dist=95;min=80;max=110
11	Температура	зі лічильником натискань кнопки	temp=21;btn_count=3
12	Освітленість	стан у вигляді бітової маски	light=200;flags=0b0101

Вимоги до звіту

Звіт повинен містити: посилання на проєкт у Wokwi, скріншот схеми підключення, повний текст скетчу, скріншот монітора порту з кількома рядками виводу у форматі вашого варіанта. Додайте короткі висновки (2-3 речення): чому millis() кращий за delay() у мікроконтролерній програмі та чому продуманий формат виводу важливий для подальшого розбору даних під час передавання мережею.

Критерії оцінювання

Оцінка	Критерій
5	GPIO, АЦП, періодична логіка на millis() та формат виводу в UART реалізовані повністю відповідно до варіанта, скетч виконується у Wokwi без помилок, студент упевнено відповідає на контрольні запитання (додаткова реалізація завдання з "Якщо впоралися раніше" не є обов'язковою умовою)
4	Функціональність реалізована майже повністю, є одна несуттєва похибка (наприклад, delay() в другорядному місці) або у звіті бракує одного з

елементів

Оцінка Критерій

3 GPIO та АЦП працюють коректно, але формат повідомлення в Serial реалізовано з суттєвою помилкою; студент розуміє загальну логіку свого коду, хоча плутається в деталях

2 Працює лише GPIO (кнопка/світлодіод), решта функціональності відсутня або реалізована формально без урахування варіанта, звіт фактично відсутній **0** Роботу не виконано, скетч не запускається у Wokwi, не відповідає варіанту, або виконаний іншою особою

Оцінки 3, 4 і 5 означають, що роботу здано. Оцінка 0-2 означає, що роботу не здано і вона підлягає доопрацюванню та повторному захисту. Робота виконується за своїм варіантом; здача чужого варіанта виявляється звірянням вихідних даних.

Контрольні запитання

1. Для чого призначений режим INPUT_PULLUP і чим він відрізняється від зовнішнього підтягувального резистора?
2. Яку розрядність має АЦП Arduino Uno і в якому діапазоні значень повертає функція analo gRead()?
3. Чому використання delay() у циклі мікроконтролерної програми вважається проблемним підходом?
4. Чому формат повідомлення, що виводиться через UART, важливо продумати заздалегідь, якщо дані згодом передаватимуться мережею?
5. У чому полягає відмінність між синхронним та асинхронним послідовними інтерфейсами передавання даних?

Домашнє завдання

Поясніть письмово, чому **delay() блокує** всю програму мікроконтролера, а побудова періодичної дії через millis() дозволяє одночасно опитувати кнопку і оновлювати ШІМ. Опишіть на прикладі свого скетчу, що саме сталося б із реакцією на кнопку, якби ви залишили delay(1000) усередині головного циклу. Також поясніть, чому вибір **роздільника** між полями (наприклад ;) важливий для того, хто згодом розбиратиме повідомлення програмою, а не читатиме його очима людини.

Без *Wokwi*, на папері, розрахуйте, яке значення поверне **10-бітний АЦП** Arduino Uno (діапазон 0-1023) для вхідної напруги 3.3 В за опорної напруги 5 В, і окремо - яку шпаруватість (%) потрібно задати в analogWrite(), щоб отримати на виході приблизно ту саму напругу.

Осмислення результату

Порівняйте поведінку delay() і millis() на прикладі власного скетчу та обчисліть значення АЦП і ШІМ, зазначені вище.

Підготовка до наступного заняття

Перегляньте Лекцію №12 (конвеєр, суперскалярність, SIMD) перед наступним заняттям.

Це не звіт з практичної роботи - звіт здається окремо. Домашнє завдання перевіряється на початку наступного заняття усно або письмово і враховується в оцінці за ту пару.